DEPLOYMENT.md

Table of Contents

Prerequisites

Required Tools

- **Node.js**: v18.x or higher (LTS recommended)
- **npm**: v9.x or higher (comes with Node.js)
- **TypeScript**: Installed as dev dependency
- **Git**: For changelog and version management features

Optional Tools

- **Mermaid CLI**: Required for diagram rendering features

```
npm install -g @mermaid-js/mermaid-cli
```

- **Playwright**: Already included as dependency, but may require system dependencies:

```
npx playwright install-deps
```

Required Accounts

- **Anthropic API Account**: For Claude AI integration

- Sign up at: https://console.anthropic.com/ - Generate API key from dashboard

Environment Variables

The following environment variables are required or recommended:

Variable	Required	Description	Example
`ANTHROPIC_API_KEY`	**Yes**	API key for Claude AI integration	`sk-ant-api03-...`
`NODE_ENV`	No	Runtime environment	`production` , `development`
`ATHENA_CONFIG_PATH`	No	Custom path to athena config file	`/path/to/athena.config.json`

Setting Environment Variables

Development (.env file):

```
# Create .env file in project root
ANTHROPIC_API_KEY=sk-ant-api03-your-key-here

NODE_ENV=development
```

Production (shell):

```
export ANTHROPIC_API_KEY=sk-ant-api03-your-key-here
export NODE_ENV=production
```

Local Development

1. Clone and Install

```
# Clone the repository
git clone <repository-url>

cd athena

# Install dependencies

npm install

# Install Playwright browsers (req

npx playwright install
```

2. Configure Environment

```
# Copy example env file (if exists) or create new
touch .env

# Add your Anthropic API key

echo "ANTHROPIC_API_KEY=sk-ant-api03-your-key-here" >> .env
```

3. Run Development Server

```
# Run with tsx (hot reload)
npm run dev


# Or build and run


npm run build

npm start
```

4. Verify Installation

```
# Test the CLI
node dist/cli.js --help


# Or during development


tsx src/cli.ts --help
```

Build

Development Build

```
npm run build
```

Output: Compiled JavaScript in `dist/` directory

Build Artifacts

- `dist/cli.js` - Main CLI entry point
- `dist/*/.js` - All compiled TypeScript modules
- `dist/*/.d.ts` - TypeScript declaration files (if configured)

Build Verification

```
# Verify build output
ls -la dist/


# Test built CLI


node dist/cli.js --version
```

Clean Build

```
# Remove old build
rm -rf dist/


# Fresh build


npm run build
```

Deployment Options

Option 1: NPM Package (Recommended for CLI tools)

Best for: Distributing as a global CLI tool

```
# Build for publication
npm run build


# Test locally


npm link
```

```
# Publish to npm (requires npm acc
```

```
npm publish
```

Installation by users:

```
npm install -g athena
athena --help
```

Option 2: GitHub Releases (Binary distribution)

Best for: Self-contained executables

Use `pkg` or similar tool to create standalone binaries:

```
# Install pkg
npm install -g pkg
```

```
# Create binaries
```

```
pkg . --targets node18-linux-x64,node18-macos-x64,node18-win-x64
```

Option 3: Docker Container

Best for: Containerized environments, consistent runtime

See Docker section below for complete Dockerfile.

```
# Build image
docker build -t athena:latest .
```

```
# Run container
```

```
docker run -e ANTHROPIC_API_KEY=$ANTHROPIC_API_KEY athena:latest
```

Option 4: Direct Repository Clone

Best for: Development teams, internal tools

Users clone and install directly:

```
git clone <repository-url>
cd athena

npm install

npm run build

npm link  # Optional: make available globally
```

Docker

Dockerfile

Create `Dockerfile` in project root:

```
FROM node:18-alpine

# Install system dependencies for P

RUN apk add --no-cache \
    chromium \
    nss \
    freetype \
    harfbuzz \
    ca-certificates \
    ttf-freefont

# Set Playwright to use installed C
```

```
ENV PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD=1

ENV PLAYWRIGHT_CHROMIUM_EXECUTABLE_PATH=/usr/bin/chromium-browser

WORKDIR /app
```

Copy package files

```
COPY package*.json ./
```

Install dependencies

```
RUN npm ci --only=production
```

Copy source and build

```
COPY . .

RUN npm run build
```

Create volume for project scanni

```
VOLUME ["/project"]
```

Set environment variables

```
ENV NODE_ENV=production
```

Default command

```
ENTRYPOINT ["node", "dist/cli.js"]

CMD ["--help"]
```

.dockerignore

Create `.dockerignore`:

```
node_modules
dist

.git

.env

.env.*

*.log

coverage

.vscode

.idea
```

Docker Compose (Optional)

Create `docker-compose.yml` for easier usage:

```
version: '3.8'

services:
```

```
  athena:
    build: .
    image: athena:latest
    environment:
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - NODE_ENV=production
    volumes:
      - ./target-project:/project
    command: ["scan", "/project"]
```

Docker Usage

```
# Build image
docker build -t athena:latest .


# Run with environment variable


docker run --rm \


  -e ANTHROPIC_API_KEY=$ANTHROPIC_API_KEY \
  -v $(pwd)/target-project:/project \
  athena:latest scan /project

# Using docker-compose


docker-compose run athena scan /project
```

CI/CD

GitHub Actions Workflow

Create `.github/workflows/ci-cd.yml`:

```
name: CI/CD Pipeline


on:


  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
```

```yaml
  release:
    types: [created]

jobs:

  test-and-build:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        node-version: [18.x, 20.x]

    steps:
      - uses: actions/checkout@v3

      - name: Setup Node.js ${{ matrix.node-version }}
        uses: actions/setup-node@v3
        with:
          node-version: ${{ matrix.node-version }}
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Install Playwright browsers
        run: npx playwright install --with-deps chromium

      - name: Build project
        run: npm run build

      - name: Verify build output
        run: |
          test -f dist/cli.js
          node dist/cli.js --version

      - name: Upload build artifacts
        uses: actions/upload-artifact@v3
        with:
          name: dist-${{ matrix.node-version }}
          path: dist/

  publish-npm:
    needs: test-and-build
    runs-on: ubuntu-latest
    if: github.event_name == 'release'

    steps:
      - uses: actions/checkout@v3

      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '18.x'
          registry-url: 'https://registry.npmjs.org'

      - name: Install dependencies
        run: npm ci

      - name: Build
        run: npm run build

      - name: Publish to NPM
        run: npm publish
        env:
```

```
          NODE_AUTH_TOKEN: ${{ secrets.NPM_TOKEN }}

  docker-build:
    needs: test-and-build
    runs-on: ubuntu-latest
    if: github.event_name == 'release'

    steps:
      - uses: actions/checkout@v3

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v2

      - name: Login to Docker Hub
        uses: docker/login-action@v2
        with:
          username: ${{ secrets.DOCKER_USERNAME }}
          password: ${{ secrets.DOCKER_PASSWORD }}

      - name: Extract version
        id: version
        run: echo "VERSION=${GITHUB_REF#refs/tags/}" >> $GITHUB_OUTPUT

      - name: Build and push
        uses: docker/build-push-action@v4
        with:
          context: .
          push: true
          tags: |
            ${{ secrets.DOCKER_USERNAME }}/athena:latest
            ${{ secrets.DOCKER_USERNAME }}/athena:${{ steps.version.outputs.
          cache-from: type=registry,ref=${{ secrets.DOCKER_USERNAME }}/athen
          cache-to: type=registry,ref=${{ secrets.DOCKER_USERNAME }}/athena:
```

Required Secrets

Configure these secrets in GitHub repository settings:

- `NPM_TOKEN`: NPM authentication token (for npm publish)
- `DOCKER_USERNAME`: Docker Hub username
- `DOCKER_PASSWORD`: Docker Hub password or access token
- `ANTHROPIC_API_KEY`: (Optional) For integration tests

Branch Protection

Suggested settings for main branch:

- Require pull request reviews
- Require status checks to pass (CI/CD workflow)
- Require branches to be up to date

Monitoring

Application Monitoring

Since this is a CLI tool, traditional APM may not apply. Consider:

1. Usage Analytics (Optional)

Track CLI usage with telemetry:

```
// Example: Add to src/cli.ts
import { track } from './utils/analytics';



// Track command usage (anonymized)


track('command:scan', { framework: detectedFramework });
```

Suggested tools:

- Mixpanel
- Segment
- PostHog (self-hosted option)

2. Error Tracking

Sentry Integration:

```
npm install @sentry/node
```

```
// src/utils/error-tracker.ts
import * as Sentry from "@sentry/node";



if (process.env.NODE_ENV === 'production') {

  Sentry.init({
    dsn: process.env.SENTRY_DSN,
    environment: process.env.NODE_ENV,
  });
}

export const captureError = (error: Error, context?: any) => {

  if (process.env.NODE_ENV === 'production') {
    Sentry.captureException(error, { extra: context });
  }
  console.error(error);
};
```

3. API Usage Monitoring

Monitor Anthropic API usage:

```
// Track in ClaudeClient class
class ClaudeClient {

  async generate(...) {
    const startTime = Date.now();
    try {
      const result = await this.client.generate(...);
      this.logUsage({
        duration: Date.now() - startTime,
        tokens: result.usage?.total_tokens,
        model: this.getModelName()
      });
      return result;
    } catch (error) {
      this.logError(error);
      throw error;
    }
  }
}
```

4. Performance Metrics

Log execution times for key operations:

```
// src/utils/metrics.ts
export const measureTime = async (name: string, fn: () => Promise<any>) => {

  const start = Date.now();
  try {
    const result = await fn();
    const duration = Date.now() - start;
    console.log([METRIC] ${name}: ${duration}ms);
    return result;
  } catch (error) {
    console.error([METRIC] ${name}: FAILED);
    throw error;
  }
};
```

5. Health Checks

For Docker deployments, add health check endpoint:

```
// src/health.ts
export const checkHealth = async () => {

  return {
    status: 'healthy',
    timestamp: new Date().toISOString(),
```

```
    dependencies: {
      anthropic: await checkAnthropicAPI(),
      playwright: await checkPlaywright(),
      git: await checkGit(),
    }
  };
};
```

Logging Best Practices

Structure logs for easy parsing:

```
// Use structured logging
console.log(JSON.stringify({

  level: 'info',
  timestamp: new Date().toISOString(),
  message: 'Scanning project',
  projectDir,
  framework: detectedFramework
}));
```

Suggested Monitoring Dashboard

Key Metrics to Track:

- Command execution frequency
- Average execution time per command
- Error rate by command type
- Anthropic API usage (tokens, costs)
- Playwright screenshot success rate
- Git operations success rate

Tools for Aggregation:

- **ELK Stack** (self-hosted): Elasticsearch, Logstash, Kibana
- **Grafana + Loki** (self-hosted)
- **DataDog** (SaaS)
- **New Relic** (SaaS)

Quick Start Checklist

- [ ] Node.js 18+ installed

- [ ] Dependencies installed (`npm install`)
- [ ] Playwright browsers installed (`npx playwright install`)
- [ ] `ANTHROPIC_API_KEY` environment variable set
- [ ] Project builds successfully (`npm run build`)
- [ ] CLI runs (`node dist/cli.js --help`)
- [ ] Optional: Mermaid CLI installed for diagrams
- [ ] Optional: Docker image built and tested

Troubleshooting

Playwright Issues

```
# Reinstall browsers with system dependencies
npx playwright install --with-deps
```

Build Failures

```
# Clear cache and rebuild
rm -rf dist/ node_modules/

npm install

npm run build
```

Anthropic API Errors

- Verify API key is correct and active
- Check API usage limits at console.anthropic.com
- Ensure billing is set up

Permission Errors (npm global install)

```
# Use npx instead of global install
npx athena <command>
```

Or fix npm permissions

```
mkdir ~/.npm-global

npm config set prefix '~/.npm-global'

export PATH=~/.npm-global/bin:$PATH
```

Related Documentation

- Readme
- Architecture
- Api
- Contributing